

Prompt Guide: Sinh Postman/Newman Test & Pact Contract Test bằng ChatGPT/Claude

Guide này dùng để copy-paste trực tiếp vào ô chat của ChatGPT hoặc Claude (bản web/app), không cần công cụ agent hay coding tool. Bao gồm 2 nhóm:

- **Nhóm I — Postman & Newman:** sinh test script JS cho API testing (functional, negative, security cơ bản, E2E).
- **Nhóm II — Pact:** sinh consumer-driven contract test thực thụ (không phải chỉ JSON Schema validation).

Các mục có nhãn **(mở rộng)** là loại test bổ sung ngoài danh sách gốc, nên cân nhắc dùng khi dự án đã trưởng thành hơn — không bắt buộc phải làm ngay từ đầu.

0. Nguyên tắc chung khi viết prompt

1. Luôn cung cấp dữ liệu thật: request/response mẫu (JSON thật), status code, headers.
2. Nêu rõ đang cần loại test nào trong danh sách bên dưới — tránh prompt chung chung "viết test cho API này".
3. Chỉ định định dạng đầu ra: script rời (dán vào tab Tests) hay cả collection JSON / cả file Pact test.
4. Với Pact: luôn nói rõ ngôn ngữ (Pact JS / Pact JVM / Pact Python...) vì cú pháp matcher khác nhau giữa các SDK.
5. Yêu cầu AI liệt kê giả định nếu thiếu thông tin, không tự bịa field.

Checklist chuẩn bị trước khi prompt

- ☐ Method + URL, headers, request/query param
- ☐ Request body mẫu, response mẫu ở happy case và error case
- ☐ OpenAPI spec (nếu có) — bắt buộc cho contract test JSON Schema
- ☐ Danh sách role/permission trong hệ thống (cho test 401/403)
- ☐ Rule nghiệp vụ đặc biệt (validation rule, unique constraint...)
- ☐ Với Pact: tên consumer, tên provider, ngôn ngữ SDK, provider state cần thiết lập

I. Nhóm Postman & Newman (API Testing)

1. Happy Path & Performance Test

Viết Postman test script cho request:

- Method: [GET/POST/ ...]
- URL: {{baseUrl}}/...
- Response mẫu (status 200/201): <dán JSON>

Yêu cầu:

1. pm.test kiểm tra đúng status code kỳ vọng (200 hoặc 201 tùy method)
 2. pm.test kiểm tra response time < 1000ms
 3. Nếu là POST/PUT, kiểm tra Location header hoặc id trả về hợp lệ
- Chỉ trả về code JS, đặt tên test có tiền tố "Functional:".

2. Schema & Type Validation Test

Response mẫu: <dán JSON>

Viết Postman test script:

1. pm.test kiểm tra Content-Type header đúng "application/json"
 2. Định nghĩa JSON Schema đầy đủ field required + kiểu dữ liệu cho response trên
 3. pm.test dùng pm.response.to.have.jsonSchema(schema) để assert
 4. Đặt tên test có tiền tố "Contract:" cho phần schema, "Functional:" cho phần header
- Nếu thiếu field nào không rõ kiểu dữ liệu, liệt kê giả định trước khi viết code.

3. Authentication & Authorization Test (401 / 403)

Endpoint: [METHOD] [URL] – yêu cầu đăng nhập và role "admin" mới được gọi.

Viết 3 Postman test case riêng biệt (3 request/3 script khác nhau):

1. "No token": không gửi Authorization header → pm.test kiểm tra status = 401, body có field error/message
2. "Expired token": gửi token hết hạn (biến {{expiredToken}}) → pm.test kiểm tra status = 401
3. "Wrong role (RBAC)": gửi {{userToken}} (role user thường, không phải admin) gọi vào endpoint admin-only → pm.test kiểm tra status = 403, không phải 401 (phân biệt rõ "không có quyền" khác với "chưa xác thực")

Đặt tên test rõ ràng: "Functional: Returns 401 when no token provided", "Functional: Returns 403 when user lacks admin role", v.v.

4. Validation & Negative Test

Request body hợp lệ mẫu: <dán JSON>
 Các field required: <liệt kê>

Viết Postman test script cho các case sau (mỗi case 1 request/script riêng):

1. Thiếu 1 field required → status 400, body chứa message lỗi nêu rõ field nào thiếu
2. Sai kiểu dữ liệu 1 field (vd: gửi string cho field number) → status 400
3. Gọi tới resource id không tồn tại (vd: GET /orders/does-not-exist) → status 404

pm.test kiểm tra cả status code và nội dung message lỗi (không chỉ status code).

5. Boundary & Input Sanitization Test (chặn SQLi/XSS)

Endpoint: [METHOD] [URL] nhận field "[tên field]" kiểu string, giới hạn độ dài [min-max].

Viết Postman test script cho các case:

1. Giá trị biên: gửi field đúng độ dài tối thiểu và tối đa hợp lệ → kỳ vọng thành công (2xx)
2. Giá trị vượt biên: gửi field vượt quá độ dài tối đa 1 ký tự → kỳ vọng 400
3. SQL Injection payload: gửi field = "' OR '1'='1" hoặc "'; DROP TABLE users; --" → kỳ vọng API KHÔNG trả lỗi 500 (không bị crash/leak), và không có dữ liệu bị ảnh hưởng ngoài ý muốn; nếu API có validate, kỳ vọng 400
4. XSS payload: gửi field = "<script>alert(1)</script>" → kỳ vọng response trả về giá trị đã được escape/sanitize (không echo nguyên payload dạng thực thi được), hoặc bị từ chối với 400

pm.test đặt tên rõ: "Functional: Rejects SQL injection payload safely", "Functional: Sanitizes XSS payload in response".

Lưu ý: đây là test kiểm tra hành vi API có xử lý an toàn không, không phải test xâm nhập thực sự – không dùng payload phá hoại trên môi trường production.

6. Data-Driven Test

Endpoint: [METHOD] [URL]

Hãy:

1. Đề xuất 1 file JSON hoặc CSV chứa ít nhất 8 bộ dữ liệu test: bao gồm case hợp lệ, case biên, case âm tính (thiếu field, sai kiểu, giá trị injection)
2. Viết Postman test script đọc dữ liệu qua pm.iterationData.get("...") và assert kết quả tương ứng với từng bộ dữ liệu (dùng cột "expectedStatus" trong data file)
3. Ghi rõ lệnh chạy bằng Collection Runner hoặc newman: `newman run collection.json -d data.csv -e env.json --iteration-count 8`

7. Integration Workflow Test (E2E — full CRUD chain)

Resource: [tên resource, vd Order]. Các endpoint liên quan:

- POST /orders (tạo)
- GET /orders/:id (đọc)
- PUT /orders/:id (cập nhật)
- DELETE /orders/:id (xóa)

Viết Postman test script cho 4 request theo đúng thứ tự chạy trong 1 folder collection:

1. POST /orders: pm.test kiểm tra status 201, sau đó pm.collectionVariables.set("orderId", jsonData.id) để dùng cho các bước sau
2. GET /orders/{{orderId}}: pm.test kiểm tra status 200 và dữ liệu trả về khớp với dữ liệu vừa tạo ở bước 1
3. PUT /orders/{{orderId}}: cập nhật 1 field, pm.test kiểm tra status 200 và field đã đổi đúng giá trị mới
4. DELETE /orders/{{orderId}}: pm.test kiểm tra status 204/200, sau đó gọi lại GET /orders/{{orderId}} và pm.test kiểm tra status 404 (xác nhận đã xóa thật)

Ghi rõ thứ tự chạy các request phải giữ nguyên (không chạy song song) vì phụ thuộc biến động {{orderId}}.

(mở rộng) Các loại test Postman nâng cao khác

8. Rate Limiting / Throttling Test

Endpoint [METHOD] [URL] có giới hạn [N] request/phút.

Viết Postman test script chạy trong Collection Runner với N+1 lần lặp: các lần đầu kỳ vọng status 2xx, lần vượt giới hạn kỳ vọng status 429, kiểm tra có header Retry-After hoặc X-RateLimit-Remaining trong response.

9. Idempotency Test

Endpoint POST [URL] hỗ trợ header Idempotency-Key.

Viết Postman test: gửi cùng 1 request 2 lần với cùng Idempotency-Key (dùng biến collection lưu key cố định), pm.test kiểm tra cả 2 lần trả về cùng 1 id/response giống hệt nhau (không tạo bản ghi trùng lặp).

10. Security Headers Test

Viết Postman test script kiểm tra response của [URL] có đầy đủ security header tiêu chuẩn: Content-Type, X-Content-Type-Options: nosniff, Strict-Transport-Security (nếu HTTPS), và nếu API public cho frontend khác domain, kiểm tra Access-Control-Allow-Origin đúng domain cho phép (không phải "*").

11. Pagination Test

Endpoint GET [URL]?page=&limit= trả về danh sách có phân trang.

Viết Postman test: gọi page=1&limit=10, kiểm tra field data.length $\leq$ 10, có field totalItems/hasNextPage;
gọi page cuối cùng, kiểm tra hasNextPage = false; gọi limit vượt giới hạn tối đa cho phép, kiểm tra API tự cap về giới hạn tối đa thay vì lỗi.

12. API Versioning / Backward Compatibility Test

API có 2 version /v1/[resource] và /v2/[resource].

Viết Postman test đảm bảo /v1/[resource] vẫn hoạt động đúng như trước (không bị breaking change)
sau khi /v2 được thêm field mới; assert các field cũ trong /v1 vẫn còn nguyên tên/kiểu dữ liệu.

II. Nhóm Pact (Contract Testing)

Khác với Postman contract test (chỉ check JSON Schema), Pact tạo ra hợp đồng `pact.json` từ chính test của consumer, sau đó dùng lại hợp đồng đó để verify provider thật — đảm bảo 2 bên luôn đồng bộ mà không cần chạy cùng lúc.

1. Consumer Interaction Test

Tôi đang viết Consumer test bằng Pact [JS/JVM/Python - chỉ định rõ] cho consumer "[TênConsumer]"
gọi tới provider "[TênProvider]" ở endpoint [METHOD] [URL].

Request mẫu: <dán JSON/param>

Response mẫu mà consumer kỳ vọng: <dán JSON>

Hãy viết Consumer Interaction Test:

1. Định nghĩa interaction với description và providerState rõ ràng (vd: "a request to get product 10", state "product 10 exists")
2. Dùng Pact Matchers thay vì hard-code giá trị cụ thể: like(...) cho field kiểu dữ liệu bất kỳ, eachLike(...) cho mảng, regex(...) cho field có format cố định (email, uuid, date...)
3. Test chạy request thật tới Pact mock server, assert response nhận được khớp với interaction đã khai báo
4. Sau khi test pass, giải thích lệnh để sinh ra file pact.json (thường tự động khi test chạy xong)

Chỉ trả về code, kèm giải thích ngắn từng matcher dùng để làm gì.

2. Provider Verification & State Test

Tôi có file pact.json (hoặc lấy từ Pact Broker) mô tả các interaction mà consumer "[TênConsumer]" kỳ vọng từ provider "[TênProvider]". Danh sách provider state cần thiết lập: <liệt kê, vd "product 10 exists">.

Hãy viết Provider Verification test bằng Pact [JS/JVM/Python]:

1. Cấu hình Verifier trở tới pact.json (local file hoặc từ Pact Broker URL)
2. Viết state handler cho từng provider state được liệt kê – mỗi handler phải thật sự setup dữ liệu (insert vào DB test/mock) trước khi Pact replay request tương ứng, và teardown sau khi xong
3. Chạy verification, để Pact tự replay toàn bộ interaction trong pact.json vào provider thật (hoặc test server) và so sánh response thật với response đã khai báo bên consumer
4. Nếu có sẵn CLI, viết luôn lệnh publish verification result lên Pact Broker

Nếu tôi chưa cung cấp đủ danh sách provider state hoặc cách setup dữ liệu test, hãy hỏi lại thay vì tự giả định.

(mở rộng) Pact nâng cao

3. Pact Broker & can-i-deploy Gating

Sau khi consumer test và provider verification đều pass và đã publish lên Pact Broker, hãy viết đoạn script/CI step dùng lệnh `pact-broker can-i-deploy` để kiểm tra xem version [X] của [consumer/provider] có an toàn để deploy lên environment [staging/production] hay không, và chặn bước deploy trong CI nếu lệnh trả về false.

4. Pending Pacts / WIP Pacts

Consumer vừa thêm 1 interaction mới vào contract, nhưng provider chưa kịp implement. Hãy giải thích và cấu hình Provider Verifier bật chế độ "pending pacts" (enablePending: true) để interaction mới này KHÔNG làm fail CI của provider ngay lập tức, nhưng vẫn hiển thị cảnh báo, cho tới khi provider verify thành công lần đầu.

5. Bi-Directional Contract Testing (BDCT)

Provider "[TênProvider]" đã có sẵn OpenAPI spec (đường dẫn/nội dung: <dán>), nhưng chưa dùng Pact theo mô hình consumer-driven truyền thống vì có nhiều consumer bên ngoài không kiểm soát được.

Hãy hướng dẫn cách publish provider contract theo mô hình Bi-Directional Contract Testing: verify OpenAPI spec bằng công cụ self-verification (vd Schemathesis/Dredd), rồi publish cả spec và kết quả self-verification lên Pact Broker/PactFlow để cross-validate với consumer contract, thay vì replay trực tiếp từng interaction.

6. Message / Event Contract Test (async, Kafka...)

Consumer "[TênConsumer]" lắng nghe message từ topic Kafka "[tên topic]", provider publish message dạng: <dán JSON message mẫu>

Hãy viết Message Contract Test bằng Pact (MessageConsumerPact/tương đương):

1. Consumer side: định nghĩa expected message dùng Matchers tương tự REST, verify consumer xử lý đúng message này
2. Provider side: viết verification test đảm bảo message provider thực sự publish ra khớp với contract đã khai báo

Bảng tổng hợp loại test (dùng để tự kiểm tra coverage)

Nhóm	Loại test	Bắt buộc	Ghi chú
Postman	Happy Path & Performance	Có	
Postman	Schema & Type Validation	Có	
Postman	Auth 401 / RBAC 403	Có	Tách rõ 2 case
Postman	Negative (400/404)	Có	
Postman	Boundary & Sanitization (SQLi/XSS)	Có	
Postman	Data-Driven	Có	
Postman	E2E Workflow (CRUD chain)	Có	
Postman	Rate Limiting	Mở rộng	
Postman	Idempotency	Mở rộng	
Postman	Security Headers	Mở rộng	
Postman	Pagination	Mở rộng	
Postman	Versioning	Mở rộng	
Pact	Consumer Interaction Test	Có	
Pact	Provider Verification & State	Có	
Pact	can-i-deploy Gating	Mở rộng	
Pact	Pending/WIP Pacts	Mở rộng	
Pact	Bi-Directional Contract Testing	Mở rộng	
Pact	Message/Event Contract	Mở rộng	

Lỗi thường gặp cần tránh

Lỗi	Hậu quả	Cách phòng tránh
Không đưa response mẫu thật	AI đoán field, sai tên/kiểu dữ liệu	Luôn dán JSON response thật
Gộp check giá trị nghiệp vụ và check schema vào cùng 1 test	Khó maintain	Tách riêng test có tiền tố Contract:/Functional:
Test SQLi/XSS bằng payload phá hoại thật trên production	Rủi ro bảo mật/pháp lý	Chỉ test trên môi trường test, dùng payload benign để kiểm tra sanitize
Nhầm 401 và 403	Che giấu lỗi phân quyền thật	Luôn viết 2 test case riêng biệt
Viết Pact Consumer test hard-code giá trị cụ thể thay vì Matchers	Contract quá chặt, dễ fail sai khi data đổi	Luôn dùng like/eachLike/regex
Provider Verification không có state handler thật	Test pass giả (false positive)	Bắt buộc handler phải setup dữ liệu thật trong DB test